

TECHNICAL DESIGN / CHECKED EXPERIMENTS

Leant2

Native Dependent-Type and Behavioral Program Synthesis in Lean

Architecture, search algorithms, correctness boundaries,
and an implementation path

Central design

Keep Lean's actual expressions, universes, local contexts, and constraints throughout search. Use a serializable search-plan layer for control, not a second approximation of Lean's type theory. Synthesize a program and its behavioral certificate together, with alternative proposals checked at one publication boundary.

Prepared for Vladimir Reshetnikov

September 17, 2026

The architecture is a proposal. The accompanying native search experiment and reference certificates were remotely compiled with Lean 4.34.0. Experimental evidence, design arguments, and unimplemented components are distinguished throughout.

Abstract

Leant2 should synthesize directly over Lean’s expressions, universes, and dependent contexts, rather than translate a Haskell-shaped type approximation. Its search-plan layer records alternatives, dependencies, costs, and replay operations without becoming a second type checker. The implementation and its behavioral proof are checked together against the original query.

This article specifies continuation-based search across correlated obligations; native head–spine construction and dependent elimination; polymorphic instantiation; structural and well-founded recursion plans; counterexample-guided refinement; finite typed solver sketches; certified completion-wide pruning; reversible observational parking; and fair scheduling. It also develops publication checks, axiom policies, exact cache identities, project-toolchain workers, implementation modules, and acceptance tests. The design preserves Leant and Djex’s evidence and ownership discipline while removing their cross-language semantic approximation.

The accompanying 178-line Lean experiment was remotely checked with Lean 4.34.0. It includes eleven whole-goal synthesized definitions, one manually chosen recursion template with synthesized branches, two output-equality checks, and two expected-failure controls. A separate checked file illustrates recursive certificates, genuine non-inhabitation, and completion-wide pruning. A Python laboratory validates five finite mechanism tests. These experiments support particular algorithms; they are not an implementation or coverage measurement of the complete architecture.

Reading guide and principal decisions

The article proceeds from native representation and search to recursion, behavior, implementation, and evaluation. Appendix A contains the tested prototype; the archive includes its sources and reproduction instructions.

Five decisions that determine the rest of the system

One semantic type representation. Lean’s Expr, universe levels, environment, and local context remain authoritative. Domain abstractions are optional views with an explicit interpretation.

Backtrack across dependent obligations. A data witness is not committed merely because it has the right type. Its remaining proof and sibling obligations must succeed, or the choice must remain revisitable.

Recursion is a typed construction. Start with library recursors and structurally justified templates, then extend to well-founded templates with explicit decreasing-call evidence.

Behavioral information has graded authority. Tests can refute concrete candidates; a proof can certify a universal specification; a completion-wide certificate can eliminate a partial program. These are different operations.

All positive results cross one check boundary. Search engines, solvers, tactics, and learned heuristics may propose. Only checked terms against the frozen original query may be published as certified solutions.

Contents

Abstract	i
Reading guide and principal decisions	i
1 Starting point: retain the semantics, replace the approximation	1
1.1 What was inspected	1
1.2 Preserve, remove, and redesign	1
2 The synthesis contract	2
2.1 A query is more than a type	2
2.2 Freeze the intended problem	3
2.3 Result types must encode the claim	3
3 Architecture: native semantics with replayable control	4
3.1 The main components	4
3.2 A native hole and its dependency closure	5
3.3 Transactions have two kinds of state	5
4 Native search: construction rules and correlated alternatives	5
4.1 Focused introduction, without premature commitment	5
4.2 The critical control rule: solve the continuation	6
4.3 Head–spine construction	7
4.4 Use native unification; do not confuse it with an oracle	7
4.5 Projection-first scheduling is a practical requirement	8
5 Universes, polymorphism, dictionaries, and dependent elimination	8
5.1 Polymorphism is native Pi structure, not unrestricted impredicativity	8
5.2 Dictionary identity is computational identity	8
5.3 Native case analysis with a dependent motive	9
5.4 Equality transport and elimination restrictions	9
6 Recursive synthesis as construction of typed algebras	10
6.1 The order of attack	10
6.2 Discover the recursion carrier from the remaining telescope	10
6.3 Generate structural plans from inductive metadata	11
6.4 Construct the output and invariant together	11
6.5 Well-founded plans and termination evidence	12

6.6	Mutual, nested, and effectful recursion	13
7	Proof search as a cooperating subsystem	13
7.1	A proof portfolio, not one monolithic tactic	13
7.2	Specification decomposition and premise selection	13
7.3	Proof cost, sharing, and output quality	14
8	Behavioral constraints as active search information	14
8.1	Combine proof-directed decomposition with example-directed discovery	14
8.2	A verifier with four outcomes	15
8.3	CEGIS without starvation or false verification	15
9	Solver-assisted completion of finite native sketches	16
9.1	What to encode	16
9.2	Dependent holes cannot be untyped multiplexers	16
9.3	Interpretation and proof reconstruction	16
10	Certified pruning of partial programs	17
10.1	The object being rejected is a family of completions	17
10.2	A sufficient abstract rejection theorem	17
10.3	A checked example: an overlong output prefix	17
10.4	When no certificate is available	18
11	Reversible observational parking and semantic diversity	18
11.1	Observational equality is provisional	18
11.2	Open and dependent terms require stricter keys	18
11.3	Diversity-aware ranking	19
12	Library discovery, indexing, and reusable contracts	19
12.1	Index for retrieval, check for applicability	19
12.2	The first useful provider catalog	20
12.3	Do not assume parametricity merely from a polymorphic type	20
13	Scheduling, caching, and the precise extent of completeness	20
13.1	Make the scheduler explicit	20
13.2	A useful conditional completeness statement	21
13.3	Cache identities must include the hidden context	21
13.4	Parallelism and memory	22

14 Negative answers and the publication boundary	22
14.1 Three negative claims that must not be conflated	22
14.2 Publication is a transaction against the original problem	23
14.3 Axiom policy and executability are separate axes	23
15 Implementation structure and user-facing integration	24
15.1 A concrete module decomposition	24
15.2 Frontends share one request model	25
15.3 Lean-native does not require one process	25
15.4 Diagnostics are part of the design	26
16 Coverage targets and an implementation sequence	26
16.1 Operationalize “a large fraction of practical Lean types”	26
16.2 Metrics and adversarial controls	27
16.3 Milestones with semantic gates	27
17 Experiments actually performed	28
17.1 Environment and verification method	28
17.2 The native search experiment	28
17.3 The initial failure and what changed	29
17.4 Checked reference certificates	29
17.5 Finite behavioral laboratory	29
17.6 What remains unimplemented	30
18 Conclusion	31
A Complete checked native-core experiment	32
B Representative production invariants and regression tests	36
C Artifact map and reproducibility	37
References	38

1 Starting point: retain the semantics, replace the approximation

1.1 What was inspected

The source baseline for Leant is commit

6bf05ad78c467989e68290f2d08bbbed40802d485.

Its `lib/Djex` submodule identifies Djex commit

e237e8667190aff38faaa590ce5b12afbfffac452.

The inspection covered the current READMEs, Leant’s synthesis-internals document and fragment translator, Djex’s architecture guide, and the earlier Lean rewrite analysis. The READMEs and architecture guide also contain historical acceptance reports. Their counts are repository evidence, not tests rerun for this article. References give the exact Leant snapshot and distinguish it from documentation read on the default branch.[1, 2, 3, 4, 5, 6]

The current system is substantially more capable than an elementary simply typed lambda-term enumerator. Djex combines a Dyckhoff-style propositional engine with a heuristic expression-search engine; its higher-rank work retains nested quantifiers, source selections, and dictionary obligations. Leant reconstructs and checks Lean candidates, and its behavioral layer maintains exact candidate/evidence associations. A successor should preserve these achievements rather than present them as newly invented features.[4, 3, 15]

The source of the architectural restriction is particularly clear in `src/Leant/Synth/Fragment.hs`. The current bridge elaborates a Lean expression, serializes a logical projection, and represents it in the Haskell search world. Its own documentation states that term-indexed applications remain opaque. Some inductive structures are expanded under restrictions, while other dependent formulas are transported as uninterpreted cargo. This can preserve sound positive reconstruction, but it prevents the search engine from using information that the original Lean type already contains.[2]

For example, an index in `Vec A (n + 1)` is not merely a label: it can rule out the empty constructor, determine a branch’s index equations, and constrain a recursive result. A `Subtype` predicate can determine which apparently interchangeable natural-number witness must be chosen. A dictionary argument can carry executable data even when another dictionary has exactly the same class type. These observations motivate a native search representation.

1.2 Preserve, remove, and redesign

The earlier rewrite report recommends pure serializable search IR and uses `MetaM` primarily at preparation and lowering boundaries. That is a defensible migration strategy for retaining the existing engines. Leant2 should take a different step: retain a pure *control and recipe* representation, but allow native metavariables within isolated branches. Serialization does not require replacing dependent types with another calculus. Deterministic replay does not require pretending that Lean unification is a pure operation.[6]

The earlier report’s runtime comparisons are not adopted as measured facts here. Branch snapshots, reference sharing, replay, and retained search frontiers need profiling under the proposed implementation. Nor does moving a metaprogram into Lean automatically make its search algorithm a theorem: a correct publication checker can keep a buggy proposer outside the logical trust boundary.

Concern	Preserve	Remove	Redesign for Leant2
Typing	Exact source identity and checked reconstruction	Haskell-kind approximation of Lean types	Native dependent constraints and levels
Search	Logical search plus ranked alternatives	Compatibility-driven engine boundaries	Shared correlated obligations and several proposal lanes
Behavior	Exact candidate ownership and counterexample replay	String/sidecar reconstruction as a semantic crossing	Direct specification terms and checked abstract summaries
Results	Budgets and qualified negative outcomes	Ambiguous “no implementation” status	Explicit typed, tested, proved, refuted, and unknown events
Execution	Project-local worker isolation	Haskell frontend and public Haskell API obligations	Lean library, tactic, commands, and version-matched worker

Table 1: The successor is a redesign, not a compatibility port.

2 The synthesis contract

2.1 A query is more than a type

Represent a fully prepared problem mathematically as

$$Q = (E, \Gamma, T, \Phi, \mathcal{G}, \Pi, B),$$

where E is the imported environment; Γ is an ordered dependent telescope of local declarations and local instances; $T : \text{Sort } u$ is the target; $\Phi : T \rightarrow \text{Prop}$ is the behavioral obligation; $\mathcal{G}$ describes permitted construction operations and providers; Π is the acceptance policy; and B is the resource budget. In type-only mode, Φ is the constantly true proposition.

A certified answer consists of terms satisfying

$$E; \Gamma \vdash t : T, \quad E; \Gamma \vdash p : \Phi(t). \quad (1)$$

For $T : \text{Type } u$, a subtype is often a convenient internal packaging. The general API should instead expose the two checked judgments: proof goals and sort-polymorphic targets need not be forced into a `Subtype`-shaped public representation.

A specification can be pointwise, relational, algebraic, or a conjunction. For example,

$$T = \text{List } \alpha \rightarrow \text{List } \alpha \rightarrow \text{List } \alpha, \quad \Phi(f) = \forall x y, f x y = x ++ y.$$

This is deliberately a component-reuse specification when `List.append` is an allowed provider. To measure algorithm discovery, remove the exact solution and equivalent high-level providers from $\mathcal{G}$, while keeping the semantic specification. A type such as $\text{List } \alpha \rightarrow \text{List } \alpha$ alone does not distinguish identity, reversal, deletion, or duplication.

Finite input/output observations belong to a separate object X , not to an implicit weakening of Φ . The user can request an example-only synthesis problem, but the result must then say that the supplied examples were checked. A universal specification is not replaced by the examples collected while searching for its implementation.

2.2 Freeze the intended problem

Elaborate the target and specification in the actual project environment before search. Preserve explicit universe parameters, binder visibility, local let-values, instance arguments, and the user-selected options. Unknown identifiers must not silently become unintended parameters in a standalone synthesis command; use a strict preparation mode and show inferred parameters explicitly.

There is a subtlety inside an existing tactic proof. A target can contain metavariables owned by its parent elaborator. The adapter must classify these as either *rigid inherited inputs* or *explicitly assignable parent obligations*. Leant2-created holes are branch-owned. A successful branch may return an assignment delta for the second category, but only through the parent transaction. It must not specialize an originally universal declaration merely to make synthesis easier. Standalone fully typed declarations should ordinarily enter search with no unresolved inherited inference holes.

The frozen query is a semantic record, not a pretty-printed string. After a candidate is found, validate it against that record, not against a goal reconstructed from the candidate's inferred type. This closes an otherwise easy loophole: a synthesizer could accidentally weaken or specialize its own request and then correctly prove the wrong thing.

2.3 Result types must encode the claim

Event or outcome	Meaning
TypedCandidate	$t : T$ has been checked; the requested universal $\Phi(t)$ has not.
ExamplesChecked	A checked typed candidate satisfies the explicitly identified finite observations. This is not a universal certificate.
Certified	Both judgments in (1) hold under the reported environment and axiom policy.
LogicalRefutation	A checked proof of $\forall t : T, \neg\Phi(t)$, or of $T \rightarrow \text{False}$ in type-only mode, under the original context.
GrammarExhausted	A precisely specified finite grammar/bound was exhausted, with the stated form of checked negative evidence. This says nothing about terms outside that grammar.
FragmentUnderivable	A complete procedure has a certificate about a reified logical fragment. Its interpretation is recorded; it is not automatically a Lean negation.
Unknown	Time, memory, proof-search, instantiation, recursion-template, or other search limits prevented a conclusion.
InfrastructureFailure	A worker, toolchain, import, protocol, or solver failed. No mathematical conclusion follows.

A query may stream several typed or tested proposals before a certified one, provided those statuses are visually and programmatically distinct. In a proof script, by `leant2` should succeed only when its assigned term closes the actual proof obligations. A command that promises a behavioral implementation should publish the implementation and its specification theorem atomically.

3 Architecture: native semantics with replayable control

3.1 The main components

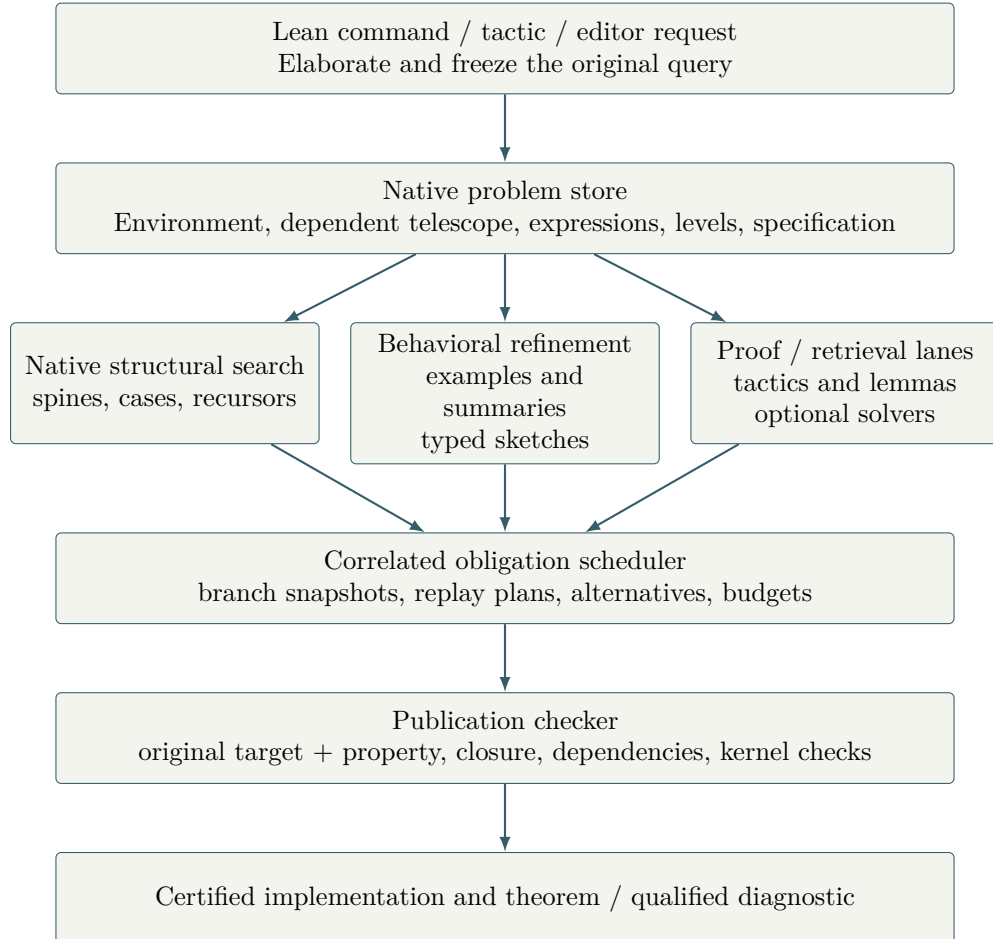


Figure 1: Proposal engines share one native problem and one acceptance path. The boxes do not denote separate type systems.

Use three distinct representations, each with a narrow purpose.

Semantic objects are Lean expressions, universe levels, declarations, and local contexts. They describe the actual problem and candidate. **Search plans** describe operations such as introducing a binder, selecting a provider occurrence, choosing a case split, or completing a recursor branch. They can be serialized after closing over stable references. **Domain abstractions** describe, for example, list lengths or bit-vector expressions. Every abstraction has an explicit connection to the semantic objects; it never replaces their typing authority.

This separation permits two execution profiles. A fast in-process profile retains native branch checkpoints. A compact frontier profile stores a checkpoint plus replay plans and reconstructs native states on demand. Both execute the same operations through the same Lean adapter. A production implementation can mix them: keep hot branches materialized and cold branches as plans.

3.2 A native hole and its dependency closure

A hole h contains an expected type T_h , a local telescope Γ_h , a metavariable identity owned by its branch, an optional residual behavioral obligation, and provenance describing how it arose. The type can depend on other holes. Its dependency set is therefore not just the free variables in T_h : include reachable metavariables in local declaration types, local let-values, postponed constraints, and the specification.

Represent the frontier as a hypergraph of construction choices and obligations. A constructor application may create several child holes and equations. Solving one child can instantiate variables appearing in the others. These children are not independent jobs merely because they have separate goal identifiers. An AND-node is an obligation to solve them under one compatible assignment; an OR-node offers alternative construction plans.

The scheduler should favor a hole that is constrained by many other obligations, that exposes a cheap constructor discrimination, or that blocks several postponed problems. A useful initial score is

$$\begin{aligned} pressure(h) = & a \text{ dependentUses}(h) + b \text{ knownIndexTests}(h) \\ & + c \text{ behaviorConstraints}(h) - d \text{ estimatedBranching}(h). \end{aligned}$$

These coefficients are heuristics, not part of soundness. Preserve deterministic tie-breaking and an independent fair lane.

3.3 Transactions have two kinds of state

Branch-local state includes the native metavariable context, pending goals, unification constraints, chosen provider occurrences, recursion hypotheses, and conditional behavioral facts. If an operation enters term elaboration, it also includes the relevant elaborator state: postponed synthetic metavariables, local instances, and diagnostics. Saving only a **MetaM** checkpoint is insufficient for an operation that mutates the surrounding elaborator or environment.

Query-global accounting is different. Rule applications, solver calls, consumed heartbeats, elapsed time, and rejected candidates must not be refunded when a branch is restored. Otherwise repeated failed choices can evade the budget. Maintain the budget owner outside backtrackable state. Trace output can be buffered per branch but must retain a monotonically increasing work identifier.

Environment changes are rare search operations. Hold generated auxiliary declarations in a branch transaction or replayable declaration plan, rather than polluting the shared environment during every proposal. Final publication checks the complete auxiliary dependency closure before committing it.

4 Native search: construction rules and correlated alternatives

4.1 Focused introduction, without premature commitment

At each goal, perform cheap weak-head reduction under a controlled transparency policy, instantiate already assigned metavariables, and expose the outer type constructor. Lean’s metaprogramming API provides the native expression, context, inference, and definitional-equality operations needed here.[7]

A dependent function goal has a canonical introduction rule:

$$\frac{E; \Gamma, x : A \vdash b : B(x)}{E; \Gamma \vdash \lambda x. b : \Pi x : A, B(x)}.$$

This introduction is suitable for eager, focused search when the objective is existence: any function inhabitant can be applied to the fresh variable to obtain a body. Preserve the binder’s visibility and the domain’s universe exactly. The same operation handles an ordinary argument, a type parameter, a dependent index, or an instance binder. There is no separate Haskell-style “type-variable introduction” calculus.

However, existence-preserving focusing does not establish that the resulting term is the best presentation or the smallest syntax tree. A provider that already has the desired function type might be preferable to its eta-expanded form. Therefore the synthesis profile may first record a cheap direct-head candidate, while the focused lane continues through lambda introduction. The quality objective and the completeness argument must not silently use different normal-form assumptions.

Products and records require similar care. Constructor-directed construction is useful, but direct local values and inexpensive projections should remain available. A sum constructor, a witness for a dependent pair, and a computational instance are genuine choices, not invertible proof steps. Their alternatives belong in the search graph.

4.2 The critical control rule: solve the continuation

Consider a context containing $a, b : \text{Nat}$ and the target

$$\{n : \text{Nat} \mid n = b\}.$$

A greedy subgoal solver may fill the witness with a , because $a : \text{Nat}$, and then become stuck on $a = b$. The appropriate algorithm restores the witness choice and tries b . This is not an exotic corner case: it is the normal interaction between dependent construction and behavioral specifications.

The core control rule is:

Listing 1: Proposed continuation-based search. Each alternative owns its complete dependent continuation.

```
search(goals, state):
  if goals are all assigned:
    return validate_residual_constraints(state)
  h, rest := choose_goal_and_dependency_order(goals)
  for proposal in resumable_proposals(h):
    checkpoint := save_branch_state()
    outcome := execute(proposal, h)
    if outcome creates children:
      if search(children together with rest, current_state) succeeds:
        return success
    restore_branch_state(checkpoint)
  return exhausted_this_frontier
```

“Together with rest” means under the same constraint store. It does not require a fixed left-to-right order. The dependency scheduler may prefer a proof equation that determines a data metavariable before it enumerates data inhabitants. The indispensable condition is that a choice remains reversible until the correlated continuation has succeeded or has been explicitly suspended with its choice point intact.

This is implemented directly in the small experiment: each native `apply` alternative calls the solver on `children ++ rest` before its transaction commits. The full engine should generalize this to a resumable continuation stack, not discard the dependency discipline when moving from depth-first to best-first search.

4.3 Head–spine construction

The primary non-introduction rule chooses a head f and constructs an application spine. Heads come from local variables, native structure projections, relevant declarations, constructors, and scoped recursion hypotheses. Given

$$f : \Pi x_1 : A_1. \Pi x_2 : A_2(x_1). \dots R(x_1, \dots, x_k),$$

freshen its universe parameters and create native metavariables for arguments as needed. Constrain the result against the expected target, then synthesize unresolved arguments in dependency order.

Do not always saturate the entire source declaration. A partially applied provider may have the desired function type; a later explicit type argument may depend on an earlier value argument; and a callback can itself be polymorphic. Enumerate meaningful spine prefixes, using the expected type to reject irrelevant ones cheaply. An explicit type argument and a value argument are both terms in a native Pi telescope, although their search policies differ.

A practical implementation uses Lean’s native application machinery for ordinary proposals, wrapped to retain all remaining goals and postponed inference obligations. It must track arguments that unification assigned automatically as well as arguments searched explicitly. A candidate occurrence is identified by the declaration or local head, its complete level instantiation, its actual ordered arguments, and its local scope. An argument assignment inferred from the result type has no less semantic importance than a displayed explicit argument.

For an unresolved computational argument, enumerate local values, projections, constructors, and recursively synthesized expressions. For a proof argument, schedule proof search. For an instance argument, first invoke Lean’s normal instance resolution under the current local context. For a type-valued argument, use demanded subexpressions of the goal and provider types, then bounded type-constructor applications and lambda abstractions. These are scheduling policies over the same native representation, not different notions of type correctness.

4.4 Use native unification; do not confuse it with an oracle

Definitional equality can instantiate metavariables. It must run inside the branch transaction. Sharing a unifier’s mutable state across alternatives is unsound as a search implementation even when the final kernel check would catch the resulting malformed candidate.

For higher-order constraints, begin with Lean’s supported unification behavior and make the residual problem explicit. When the unifier cannot resolve a flexible family application, consider a finite set of typed instantiations: projections, constant families, available type constructors, or lambdas obtained from the demanded target shape. Each is a branch. Do not replace a failed or resource-limited unification attempt by a theorem that two arbitrary dependent expressions are unequal.

In particular, distinguish three outcomes in the adapter: a successful assignment; a certified or syntactically decisive incompatibility within the attempted rule; and an unresolved constraint requiring a different transparency setting, additional arguments, or further search. The last is not a reason to cache global non-inhabitation.

The first-order simplification layer should also wake constraints when their dependencies change. Attach a dependency stamp to postponed work; retry after a relevant metavariable assignment, not after every unrelated step. This avoids both premature rejection and repeated expensive unification of an unchanged problem.

4.5 Projection-first scheduling is a practical requirement

Our first prototype tried constructors before discovering structure fields. A target natural-number witness constrained to equal the value in the second of two dictionaries caused constructor growth and eventually exhausted Lean’s heartbeat limit. Adding exact structure-projection proposals before constructor expansion made the final test file compile successfully.

This does not prove that projections should always dominate every constructor. It establishes a concrete failure mode for a naive priority policy. The default proposal order should be cheap local heads, constraint-directed reflexivity, inexpensive projections, relevant library heads, bounded constructors, and then increasingly expensive elimination and recursion plans. The fair lane may explore other orders, but the heuristic lane should not construct large numerals before inspecting an already available field of the required type.

5 Universes, polymorphism, dictionaries, and dependent elimination

5.1 Polymorphism is native Pi structure, not unrestricted impredicativity

A callback of type

$$\forall \alpha : \text{Type } u, \alpha \rightarrow \alpha$$

is synthesized by introducing the type parameter and then the value parameter. Constructing such a callback as an argument to another function does not need a Haskell rank-N approximation. The same rule composes dependent callbacks of type $\forall x, P(x) \rightarrow Q(x)$.

Native representation does not remove Lean’s universe discipline. A polymorphic function type quantified over `Type u` can live at a higher universe than the types it quantifies over. Leant2 must not instantiate a parameter expecting a smaller universe with that type merely because a System-F-like encoding would permit it. Keep `Level` parameters and constraints; preserve the distinction between proposition-valued products and data-valued products; and do not assume an implicit universe-subtyping or lifting rule.[8]

Fresh universe metavariables are allocated for each provider occurrence, while named universe parameters in the original query remain rigid. Two occurrences of a polymorphic provider may receive different instantiations. Conversely, a single occurrence with correlated arguments must share its own equations. A target involving two independent universes is not solved by silently identifying them. The accompanying experiment checks a small two-universe callback example, not the original larger two-universe Church-composition regression from Leant’s repository.

5.2 Dictionary identity is computational identity

For a class with a field `value : A`, two values $d_1, d_2 : C\ A$ can contain different data. Their types being definitionally equal does not justify substituting one dictionary for the other. Store the actual dictionary expression at every method occurrence. A context key containing only the class predicate is insufficient.

Use Lean’s ordinary local-instance and type-class behavior as the default policy. This preserves the meaning of a user-written method call. Optionally allow explicit synthesis over multiple dictionary values, but describe that as a construction choice, not as an alternate hidden instance-resolution semantics. The output must retain the chosen dictionary even if a pretty printer would ordinarily omit it.

Likewise, **Decidable P** lives in the computational world and contains a choice between two constructors. It is not erased merely because it carries proofs. Proof irrelevance applies to proofs of a fixed proposition; it does not identify arbitrary data structures containing proofs. The successful `chooseDictionary` experiment checks the actual selected payload with a definitional equality theorem.

5.3 Native case analysis with a dependent motive

For a local value $x : I \vec{p} \vec{i}$, obtain the inductive declaration and recursor metadata from the environment. The search adapter should initially delegate the difficult details of dependent case analysis to Lean’s native case-analysis machinery, which produces the branch contexts and goals. A custom high-performance path can be added later against differential tests; it should not be the first implementation of dependent elimination.[9]

The elimination plan has four explicit steps. First, find every local declaration and target fragment whose type depends on the scrutinee or indices. Second, generalize the necessary dependent suffix of the context into the motive. Third, create constructor branches and specialize the motive using the actual constructor arguments and index equations. Fourth, search each branch under its resulting context and reconstruct the eliminator application.

An impossible branch is removed only when the native construction proves its impossibility: incompatible constructors, contradictory indices, or a checked local contradiction. A syntactic mismatch between an opaque index expression and a constructor result is not enough. When normalization does not decide an arithmetic relation between indices, generate a proof obligation or select another eliminator plan.

For the target $\text{Vec } \alpha (n + 1) \rightarrow \alpha$, the empty constructor cannot describe the input. The surviving `cons` branch introduces a head of type α , which solves the result. No conversion to a non-indexed list type is necessary. The prototype exercises exactly this mechanism.

5.4 Equality transport and elimination restrictions

When the context contains $e : a = b$ and $v : P(a)$, the desired $P(b)$ need not be definitionally equal to $P(a)$. A valid proposal performs equality elimination or introduces a checked transport. Record the equality proof and the motive; do not just change an expression’s stored type after an SMT equality result.

Proposition-valued inductives require their actual Lean elimination rules. An existential proposition or **Nonempty A** is not interchangeable with a computational dependent pair. Extracting an arbitrary executable witness from such evidence may require classical choice and a noncomputable definition. Leant2 should offer that route only under an explicit policy and label it accordingly, rather than silently converting proof search into computational synthesis.[9, 12]

For records, indexed structures, and families with dependent fields, preserve constructor telescopes in order. A later field can constrain an earlier field even when the earlier field is not a proof. The continuation mechanism of Section 4 is therefore the general rule for both ordinary programs and certified data structures.

6 Recursive synthesis as construction of typed algebras

6.1 The order of attack

Recursive synthesis should have three progressively broader routes: reuse an existing iterator or combinator; construct a structural recursor application; then synthesize a well-founded recursion plan. This order is motivated by search size and certificate reuse, not by a restriction that all interesting programs must be folds.

Lean already translates structural recursion into recursor-based definitions and supports well-founded recursion justified by decreasing relations. Structural definitions generally offer stronger definitional computation, while well-founded definitions can require propositional unfolding equations. Leant2 should exploit those existing construction and checking paths rather than invent an unchecked recursive term constructor.^[10]

The untrusted search implementation itself may use `partial def`, as the prototype does. That is different from allowing a generated implementation to bypass the user’s totality and behavioral requirements. A partial search routine proposes ordinary checked terms; it is not inserted into the computational result.

6.2 Discover the recursion carrier from the remaining telescope

Suppose the desired function has the shape

$$\text{List } \alpha \rightarrow \text{Nat} \rightarrow \text{Option } \alpha.$$

A list fold with carrier $\text{Option } \alpha$ is too restrictive for general indexing. A fold with carrier

$$C = \text{Nat} \rightarrow \text{Option } \alpha$$

is appropriate. Its step receives an element and a function describing lookup in the tail:

Listing 2: A reference fold template for zero-based lookup; proposed search can discover the carrier from the target telescope.

```
fun xs =>
  List.foldr
    (fun a smaller i =>
      match i with
      | 0 => some a
      | n + 1 => smaller n)
    (fun _ => none)
    xs
```

This displayed template is an explanatory construction, not a claim that the attached prototype discovers it. It illustrates a specific algorithmic improvement over searching only first-order fold results: group the residual argument telescope into the carrier before instantiating the fold provider. Result-type unification then propagates that carrier into the base and step obligations.

Maintain binder provenance even after eager introductions. When considering recursion on `xs`, the engine must be able to reabstract `i`; otherwise it has only an induction hypothesis about the original fixed index, which cannot be used at `n` in the successor branch. Generalization therefore includes more than the transitive *type* dependency closure. It must also consider arguments whose values change across recursive calls.

A concrete carrier-discovery policy is: first reabstract all remaining non-fixed arguments in their original order; then try a small family of subsets suggested by the specification and by candidate recursive calls. Include mandatory type dependencies whenever a selected binder requires them. Bound the number of subsets and retain the general plan as a fallback. Do not guess carriers by enumerating unrelated global types.

6.3 Generate structural plans from inductive metadata

For each eligible input, obtain its actual recursor type and inspect the constructor methods. The general algorithm is:

Listing 3: Structural-plan generation. All method holes retain their native dependent types.

```
for major in eligible_inductive_inputs:
  for generalization in telescope_plans(major, target, specification):
    motive := abstract_target_and_contract(major, generalization)
    recursor := select_native_recursor(major, motive.sort)
    plan := instantiate_recursor(recursor, motive, parameters, major)
    expose method holes and native induction hypotheses
    schedule search over all methods and shared constraints
    if a complete candidate survives:
      submit the reconstructed recursor term to publication checking
```

In a constructor branch, recursive hypotheses are introduced by the recursor, with the precise types dictated by the constructor’s recursive fields. For a branching tree there can be several; for a function-valued recursive field there can be a dependent family of hypotheses. Do not flatten all of these into an untyped “smaller result” array.

The motive can depend on indices and the major value itself. A function producing $\text{Vec } A \ n$, for example, has a result family indexed by the natural-number argument. The reference experiment’s `vectorReplicate` uses a hand-selected induction on n ; the synthesizer fills the base and successor cases under their native types. A full recursion planner should automate exactly that template choice and branch formation.

Repeated case analysis is not a substitute for recursion. Bounded unfolding can satisfy examples up to a certain input size without yielding a generally correct recursive implementation. The grammar must mark whether a plan contains a genuine recursor, a bounded unrolling, or a nonrecursive term, and the verifier must check the corresponding actual definition.

6.4 Construct the output and invariant together

Let D be an inductive input type, $B(x)$ the output family, and $R(x, y)$ the required behavior. Rather than first guessing

$$f : \prod x : D, B(x)$$

and later attempting an unrelated global proof, construct an algebra for

$$C(x) = \{y : B(x) \mid R(x, y)\}. \quad (2)$$

Each induction hypothesis then carries a smaller output and its correct local specification. The step can use both. For an additional argument telescope, place that telescope inside C in the appropriate dependent order.

Here is the exact recursive construction checked in `Certificates.lean`:

Listing 4: Checked reference construction, authored manually. The implementation and proof follow the same recursion.

```
def certifiedAppend  $\alpha\{ : \text{Type } u\}$  (ys : List  $\alpha$ ) :
  (xs : List  $\alpha$ )  $\rightarrow$  {zs : List  $\alpha$  // zs = xs ++ ys}
| [] => (ys, ) rfl
| x :: xs =>
  let tail := certifiedAppend ys xs
  (x :: tail.val, CongrArg (List.cons x) tail.)property
```

The significant search object is the branch algebra, not the final text. In the cons branch, the target equation suggests the outer constructor, and the induction hypothesis supplies the needed equation for the tail. The computational hole and the proof hole constrain one another immediately.

Proposition 6.1 (Certificate-algebra composition). *Suppose a native recursor has motive C from (2), and all of its method arguments are checked terms of the method types. The resulting recursor application supplies an output satisfying R for every input in its domain.*

Proof. The recursor application is a term of $\Pi x : D, C(x)$ by its type. At each x , the first projection has type $B(x)$ and the second projection proves R of that first projection. No separate induction on the generated source text is required. The claim is conditional on using the actual checked recursor and method terms, not on trusting the plan generator. $\square$

There are important limits. A specification relating several recursive calls, prescribing an accumulator invariant, or comparing several outputs may not fit a useful motive without strengthening it. Treat invariant strengthening as another bounded synthesis problem: propose a generalized relation from library lemmas and symbolic execution, then require the base and step obligations to be proved. Do not add an invented invariant as an axiom.

6.5 Well-founded plans and termination evidence

For a well-founded relation r on the input domain, a recursion step has access to a hypothesis of the form

$$rec : \forall y, r(y, x) \rightarrow C(y).$$

This is a precise interface for synthesizing recursive calls: a proposed call at y creates a proof obligation $r(y, x)$. The function under construction is not simply offered as an unrestricted provider of type $\forall y, C(y)$.

The first measure grammar should include natural-number arguments, lengths, sizes, bounded differences, lexicographic combinations, and registered domain-specific measures. Each candidate relation needs a well-foundedness theorem. An arbitrary integer order is not a well-founded measure just because a sampled execution decreases. Under a branch guard, arithmetic tactics may prove that an index-distance measure decreases; absent the guard, the same proposed recursive call may be invalid.

Synthesize in two phases within one revisitable plan: propose a computational call structure and measure, then discharge each local decrease obligation and the behavior step. Failure to prove a decrease should return to the measure or call choice, not be converted into a declaration of divergence. The final native definition or explicit `WellFounded.fix` application remains subject to normal checking.

A useful implementation boundary is to retain a `TerminationPlan` containing the chosen relation expression, its well-foundedness proof, and each recursive occurrence's argument and decrease evidence.

The expression `lowerer` constructs the native recursion term from this object. Source rendering may use `termination_by`, but the accepted term and evidence are the authority.

6.6 Mutual, nested, and effectful recursion

The same plan structure can later support mutual recursion by working over a tagged dependent sum of the input domains or by delegating a mutually recursive declaration plan to Lean’s elaborator. Nested datatypes and course-of-values recursion should use their actual generated recursion principles. These extensions require more elaborate plan generation, not a new acceptance boundary.

For effectful code, a type such as `IO A` says almost nothing about the external behavior. Use a registered operational or Hoare-style specification, identify the modeled effects, and require the appropriate verification conditions. Merely synthesizing a term of the monadic type is type-only synthesis. Similarly, partial-fixpoint and coinductive constructions need their own semantic profiles; they must not be admitted into a total-function benchmark without changing the stated contract. Lean’s modern recursion facilities include these distinct mechanisms, but support for them is a later extension of this design.[10]

7 Proof search as a cooperating subsystem

7.1 A proof portfolio, not one monolithic tactic

Use several proof-producing operations behind a small adapter: reflexivity and local assumptions; simplification with an explicit lemma set; arithmetic procedures such as `omega`; congruence and instantiation through `grind`; and a configurable general proof-search lane such as Aesop. Bit-vector goals can be routed to a verified decision procedure available in the pinned toolchain. These tools are components, not the definition of the Leant2 search strategy. Lean’s documented `grind` interface supplies an existing extensible automation path.[11, 14]

Each invocation has an obligation, a branch checkpoint, an allowed lemma environment, a budget, and a result classification. A tactic that solves a proof goal may instantiate computational metavariables appearing in it. Such assignments must return as part of the branch’s construction choice. A “proof-only” lane that secretly fixes a data witness can otherwise reintroduce the same greedy commitment bug that continuation search was designed to avoid.

An inexpensive first policy is to delay heavyweight tactics until computational metavariables in the goal are either assigned or explicitly exposed as permitted witness choices. Use symbolic simplification earlier when it can reveal constraints such as $n = b$ without choosing a large search proof. Preserve equations that guide construction, rather than normalizing away all provenance needed for explanation.

7.2 Specification decomposition and premise selection

For a conjunction, create linked obligations but keep common computational parameters shared. For a universal property, introduce symbolic inputs and retain their relation to example generators. For a conditional property, place the precondition in the proof context and in the concrete test-input admissibility check. For an existential property of an already fixed result, witness synthesis becomes an ordinary dependent construction problem.

A library theorem can serve as a sufficient contract for a provider. Applying it first may convert a global behavioral problem into local argument obligations. This backward proof search and forward program

construction should meet at the same provider occurrence. The proof must refer to the selected type arguments, dictionary values, and executable function, not to a theorem about a superficially similar overload.

For example, synthesizing a sorting routine from a comparison interface requires whatever order laws the chosen theorem assumes. The mere presence of a comparator does not establish transitivity or totality. Expose those laws as actual premises or choose a weaker specification. This rule prevents a common synthesis error: treating a familiar class name as a mathematical guarantee not present in its Lean type.

7.3 Proof cost, sharing, and output quality

Track program cost and certificate cost separately. A short program whose proof expands to an enormous decision trace may be less desirable than a slightly longer implementation with a small reusable lemma. Conversely, penalizing all proof terms by raw size can reject good proof-erased implementations for the wrong reason.

After finding a solution, perform checked simplification: eliminate unnecessary lets, replace redundant casts with proved equalities, and reuse lemmas. Recheck the simplified term and specification against the original query. Proof sharing through named auxiliaries is valid only when those auxiliaries are included in the same dependency audit. A small printed term referring to an enormous helper is not a genuinely small artifact unless the cost model explicitly counts library reuse that way.

8 Behavioral constraints as active search information

8.1 Combine proof-directed decomposition with example-directed discovery

The specification should influence construction before complete terms are enumerated. Three kinds of information are useful: logical obligations that Lean can prove or simplify; concrete observations on admissible inputs; and abstract summaries that approximate the behavior of all completions of a partial program.

Refinement-type synthesis demonstrates the value of decomposing specifications while constructing programs. Type-guided abstraction refinement and abstraction-guided example synthesis offer complementary ways to scale component search. Leant2 should borrow those strategies, while retaining Lean’s native dependent typing and requiring checked interpretations for any pruning claimed to preserve solutions.^[16, 17, 18]

A backward contract step selects a provider theorem whose conclusion can imply the desired property and generates obligations on its arguments. A forward step symbolically evaluates a proposed expression or computes its abstract summary. The two meet through the provider’s exact instantiated application. For a cons-producing branch, for example, an output-length requirement becomes a condition on the tail; an output equality can expose the required head and tail equations.

For higher-order functions, pass behavior to a callback hole only when the propagation is justified. A map specification with concrete list inputs can generate concrete callback observations at those elements, but a relational property such as injectivity is not generally determined by those observations. A fold may require an invariant relating the accumulator to the processed prefix. Preserve the difference between a useful example projection and a sufficient logical contract.

8.2 A verifier with four outcomes

For a complete typed candidate, use the interface

$$\text{verify}(t, \Phi) \in \{\text{Proof}(p), \text{Counterexample}(c), \text{Unknown}(r), \text{Failure}(e)\}.$$

A proof is checked against $\Phi(t)$. A counterexample records an admissible input, any required path/pre-condition evidence, the relevant evaluation, and a checked contradiction to the candidate's requested behavior. An unknown means that the attempted method did not decide the obligation. An infrastructure failure is not mathematical evidence.

For an open problem, a concrete model can refute the explicitly abstracted universal closure of the candidate. This must not be confused with a proof that the candidate fails at every possible instantiation of the local context. Preserve the closure, the substituted type parameters, and witnesses for all instantiated premises. A solver cannot conjure an element of an arbitrary empty type merely to make a behavioral counterexample convenient.

Untrusted native execution is useful for discovery and ranking. In certified mode, a proposed failing observation must be replayed through an approved logical evaluator, a checked decision result, or an appropriate proved correspondence theorem before it justifies an irreversible semantic rejection. Timeouts, crashes, and unknown outputs are not values in the observation domain.

8.3 CEGIS without starvation or false verification

The basic counterexample-guided loop is familiar, but two details matter enough to make explicit: verification can remain unknown, and later candidates must still receive attention.

Listing 5: Proposed certificate-aware CEGIS coordinator.

```
examples := initial_examples
pending_proofs := fair_queue()
while resources remain:
  choose fairly between generation, pending proof work, and refinement
  candidate := obtain a closed typed proposal
  if a checked example refutes candidate:
    record the counterexample; refine compatible frontiers; continue
  outcome := verify_under_a_budget(candidate, original_specification)
  match outcome:
    Proof(p): publish only after original-query kernel validation
    Counterexample(c): replay c; refine examples and completion summaries
    Unknown(reason): retain candidate and schedule other work
    Failure(error): report operational failure; do not infer falsehood
```

A fixed set of examples constrains the next generation round. New examples are applied to candidates and partial-program summaries whose context and domain stamps are compatible. Do not apply a counterexample from one dictionary selection or universe specialization to an unrelated problem merely because its printed function name is the same.

When a candidate passes every currently available test, its status remains tested. It becomes certified only after a proof is reconstructed. For a genuinely finite domain, exhaustive checking can establish a universal property, but only with a completeness theorem for the enumerator and a checked decision procedure for the property. An arbitrary bounded list generator is not such an enumerator for all lists. The candidate generator should also continue when an external solver returns `unknown`. Otherwise the

first difficult but unproved candidate can block a later, easily certified one indefinitely. Verification budgets can grow on revisits, while their consumed work remains charged to the global budget.

9 Solver-assisted completion of finite native sketches

9.1 What to encode

SMT is most useful when a large combinatorial choice has a small explicit shape: selecting operators, choosing among available arguments, selecting a branch predicate, or filling arithmetic constants in a bounded sketch. It is not a replacement for Lean’s elaborator or dependent type checker.

A sketch consists of a finite grammar of native construction alternatives, selector variables, typed holes, and a mapping from each selector assignment to a candidate reconstruction plan. Generate a solver problem from concrete examples or from a proved abstract semantics of that sketch. Decode a model into choices, replay those choices in a fresh native branch, and check the resulting Lean term. The solver is permitted to produce a proposal that fails reconstruction; it is not permitted to declare that proposal well typed.

Type-directed pruning can greatly reduce the selector problem before any solver call. First-order input and output shapes, permitted operations, and explicit finite alternatives are suitable abstractions. TY-GAR motivates using coarse type information to avoid exploring the entire component graph at once; in Leant2, an abstraction miss triggers refinement and wider native retrieval rather than a change to the meaning of the requested type.^[17]

9.2 Dependent holes cannot be untyped multiplexers

For a dependent pair, selecting a witness changes the type of the second field. Suppose one branch chooses $n = 0$ and another chooses $n = 1$, producing second-field obligations in different fibers. Encoding both as a single untyped solver variable and later attaching the chosen type is invalid.

There are two sound construction strategies. In a branch-expanded encoding, each witness alternative owns its correctly specialized child grammar, guarded by its selector. In a native-first encoding, the solver selects only the witness; Lean then generates the dependent residual problem and a later solver query chooses its completion. The former can exploit larger simultaneous constraints; the latter avoids an exponential family of heterogeneous child grammars. Both reconstruct the same kind of native term.

Similarly, do not ask an SMT engine to invent arbitrary Lean universe expressions or inhabitants of unknown higher-order types. Keep those choices native and bounded, or provide a domain-specific reification with explicit soundness obligations. A solver-level uninterpreted sort named after a Lean type is not itself evidence about that type’s inhabitants.

9.3 Interpretation and proof reconstruction

Every solver domain specifies its representation map and the meaning of its constraints. For natural numbers represented by integers, include nonnegativity. Truncated natural subtraction is not integer subtraction without conditions. Fixed-width bit-vector operations must use the correct modular semantics. Division, casts, comparisons, and overflow behavior need explicit treatment, including exceptional or zero-divisor cases supported by the Lean definitions.

The model path and the proof path are separate. A model proposes a program or a counterexample, both replayed in Lean. An `unsat` answer is useful for heuristic ranking immediately, but it supports certified pruning only after a trusted translation theorem and checked unsatisfiability evidence are available. Depending on the domain, that evidence may be reconstructed by an existing Lean tactic or by a small reified checker. A raw solver string never becomes a proof term.

A nogood learned from a failed candidate must identify the exact choices it rules out. Generalizing it to other choices requires a certificate or keeps it heuristic. Do not cache a model-based conflict across changed assumptions, different bit widths, changed dictionary values, or different instance environments.

10 Certified pruning of partial programs

10.1 The object being rejected is a family of completions

For a partial program h , let $Comp(h)$ be its legal native completions under the current grammar and inherited constraints. A completion may itself be a dependent tuple: later hole assignments can depend on earlier ones. Let $assemble(h, c)$ be the complete expression obtained from completion c .

An irreversible semantic rejection of h is justified by evidence of

$$\forall c : Comp(h), \quad \neg \Phi(assemble(h, c)). \quad (3)$$

A counterexample to one completed term is not evidence of (3). For example, the sketch `append xs ?tail` has an incorrect completion `?tail := []` and a correct completion `?tail := ys` for the `append` specification. Rejecting the whole sketch because the first completion failed would lose the solution.

10.2 A sufficient abstract rejection theorem

Let $\sigma(t)$ describe a complete candidate in an abstract domain. Let A_h overapproximate the summaries of all legal completions, and let N_Φ be a necessary abstract condition for success:

$$\forall c : Comp(h), \quad \sigma(compile(h, c)) \in A_h, \quad \Phi(t) \implies \sigma(t) \in N_\Phi.$$

Proposition 10.1 (Completion-wide pruning). *If the two implications above are justified and $A_h \cap N_\Phi = \emptyset$ is proved, then (3) follows.*

Proof. For an arbitrary legal completion c , its summary lies in A_h . If its assembled term satisfied Φ , that summary would also lie in N_Φ , contradicting disjointness. Since c was arbitrary, every legal completion is ruled out. $\square$

The direction of approximation matters. An underapproximation can show that some attractive completions fail, but emptiness of its intersection cannot eliminate the unrepresented completions. Likewise, an abstract model must describe a realizable input before it can become a concrete counterexample. These obligations should be part of each domain plugin's interface, not comments left to its implementer.

10.3 A checked example: an overlong output prefix

Suppose a partial list program has already committed to a fixed output segment and can only append more elements. For an admissible input whose length is smaller than that segment's length, no suffix can make the output have the input's length. The accompanying file checks precisely this theorem:

Listing 6: Checked completion-wide rejection theorem.

```

theorem prefixCannotComplete  $\alpha\{ : \text{Type } u\}$  (input fixedPart : List  $\alpha$ )
  (tooLong : input.length < fixedPart.length) :
   $\forall$  suffix : List  $\alpha$ , (fixedPart ++ suffix).length  $\neq$  input.length := by
intro suffix equality
simp only [List.length_append] at equality
omega

```

The theorem applies even if the suffix hole has additional constraints, because it quantifies over all suffixes. But it only applies to a grammar position that really preserves the committed segment by append. A later operation allowed to delete or replace that segment invalidates the summary assumption. The rejected family’s grammar identity is therefore part of the certificate’s applicability check.

This proof depends on `propext` and `Quot.sound` in the tested toolchain, as reported by the axiom audit. It does not depend on `sorryAx`. A stricter axiom profile may select a different arithmetic proof; the published certificate must disclose which policy actually accepted it.

10.4 When no certificate is available

An abstract analysis can still rank, defer, or selectively explore a branch without claiming a theorem. Such heuristics belong in a lane whose omissions are compensated by a fair retained baseline. Store the reason as `HeuristicDeprioritization`, not `CertifiedReject`. The positive publication guarantee remains intact either way, but only certified rejection or faithful replay preserves the corresponding search-coverage claim.

11 Reversible observational parking and semantic diversity

11.1 Observational equality is provisional

For concrete inputs $X = \{x_1, \dots, x_k\}$, define the observation vector

$$obs_X(t) = (eval(t, x_1), \dots, eval(t, x_k)).$$

Many different candidates share this vector. Storing only the cheapest representative can save enormous work, but destroying the others can lose the desired implementation. The empty-list function and identity agree on an empty input and disagree on a singleton input.

Use *parking* instead: a bucket holds an active representative together with its other members or exact regeneration recipes. When a new example arrives, recompute or extend observations and split the bucket. Newly distinguished members become eligible again. The recipe must retain the original context and grammar choices; a vague promise to “enumerate later” is not a complete retention mechanism.

There is a second liveness requirement: rotate members fairly even when no new counterexample arrives. The active representative may remain hard to prove, while a parked equivalent-looking candidate has a short certificate. Waiting forever for a distinguishing test would block the latter. Proof-search diversity is therefore a reason to reactivate members independently of observational splitting.

11.2 Open and dependent terms require stricter keys

Do not observationally merge arbitrary open expressions. A higher-order term may later be placed in a context that distinguishes it; a dependent term may live in a fiber whose index is still unassigned; and

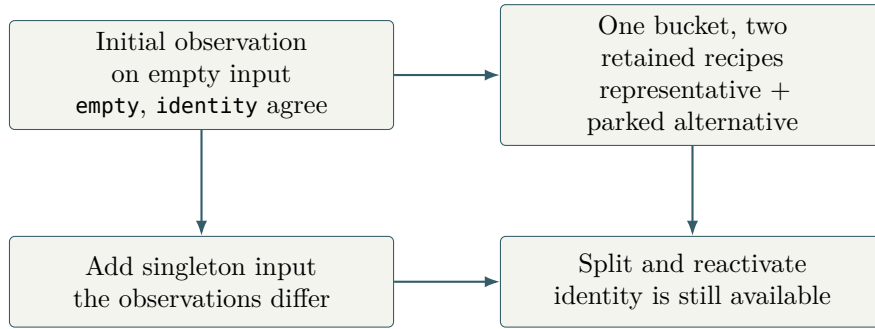


Figure 2: A finite-test equivalence class is a scheduling device, not a proof that its members are interchangeable.

an apparent observation may depend on an unresolved dictionary value.

The initial implementation should restrict parking to closed typed candidates, or to closures over the exact same frozen telescope with the same admissible instantiations and observed result family. Include evaluation status in the bookkeeping, but never equate two timeouts with equal returned values. Dependent outputs need a well-typed comparison operation in the same fiber or an explicit transport into a common observation domain.

When a Lean proof establishes an appropriate equality, stronger merging is possible. For first-order values, a checked equality permits substitution in the specification. For functions, a pointwise equality plus the appropriate extensionality principle can justify the required replacement. This is distinct from finite observation equality and should be represented by a different object in the API.

11.3 Diversity-aware ranking

A useful result stream balances small source size, low proof cost, inexpensive computation under an explicit cost model, and semantic diversity. Prefer a representative from an unexplored behavior bucket over the tenth syntactic variant of the current best candidate. Keep the final score separate from any admissible lower bound used by a completeness-oriented scheduler.

Resource-aware synthesis research shows how a specified cost semantics can guide construction. Leant2 can support such a profile by attaching checked resource bounds to providers and recursive plans. A source-level cost certificate is not a theorem about actual wall-clock performance of compiled code; its operational interpretation must be stated separately.^[19]

12 Library discovery, indexing, and reusable contracts

12.1 Index for retrieval, check for applicability

A practical synthesizer cannot scan every imported declaration at every hole. Build persistent environment indexes keyed by coarse result heads, argument heads, constructor families, theorem conclusion shapes, and selected semantic tags. Use a discrimination-tree-style index for structural matches and a secondary graph linking providers through input/output type abstractions. Local values and projections form a separate small inventory searched first.

These indexes return candidates, not proof of applicability. Native instantiation and checking decide whether a returned provider fits. Expand retrieval in stages: exact apparent result head, reducible

aliases, more general indexed patterns, related input/output heads, and an explicit wider scan under a budget. A failed narrow index query must not become negative inhabitation evidence.

Cache immutable declaration metadata with the environment generation. Cache an instantiated occurrence only with its complete arguments, universes, and inherited constraint stamp. Never reuse a theorem about one local dictionary payload for another merely because both methods have the same printed notation.

12.2 The first useful provider catalog

The initial catalog should cover constructor/projection plumbing, option and sum combinators, list and array iteration, arithmetic and finite-index utilities, function composition, common record transformations, and their elementary specification lemmas. The catalog does not contain a new abstract model of these types; it records native declarations and how their roles can guide search.

Annotations can describe a provider’s intended search role, cost estimate, legal unfolding policy, corresponding contract lemmas, and available abstract summaries. For example, a list operation may have a checked length theorem and a content theorem. A proof theorem can be marked as a likely backward rule for a target relation. These are modular additions; users should not have to modify the synthesizer to support a project datatype.

A summary record must identify the theorem that justifies it and a procedure that instantiates that theorem at the *same* provider occurrence used by the candidate. If no checked theorem is attached, the summary is explicitly heuristic. Opaque library implementations remain useful through their interfaces and theorems; aggressive unfolding should not be a prerequisite for reuse.

12.3 Do not assume parametricity merely from a polymorphic type

Some Haskell-inspired synthesis heuristics are motivated by free theorems. Lean’s standard axioms permit nonparametric polymorphic functions, so unrestricted parametricity is not a valid global assumption. The Lean reference gives an explicit discussion of this distinction.[12]

A parametricity-based pruning domain must be tied to a restricted grammar and a proved logical relation for that grammar, or to a theorem about the particular provider. It may also be used as a nonbinding heuristic. The type $\forall \alpha, \text{List } \alpha \rightarrow \text{List } \alpha$ alone is not authorization to import a Haskell semantic theorem into every Lean context.

13 Scheduling, caching, and the precise extent of completeness

13.1 Make the scheduler explicit

Use resumable jobs rather than an implicit lazy stream hidden inside one search function. A job records a query generation, branch identifier, plan cursor, pending continuation, priority components, and the resources already spent. Its kind identifies native expansion, provider retrieval, case planning, recursion planning, proof work, or behavioral refinement.

Each scheduling quantum executes one bounded primitive operation or a bounded group of cheap operations. Native checkpoints are taken at well-defined operation boundaries. An arbitrary tactic invocation cannot necessarily be suspended halfway and serialized; on timeout, restore its checkpoint and either retire that attempt or retry with a larger budget later. Retried work is charged again.

Use a heuristic priority such as

$$s(j) = g(j) + \lambda_h \hat{h}(j) + \lambda_p \hat{p}(j) + \lambda_d \text{duplication}(j) - \lambda_r \text{relevance}(j),$$

where g measures constructed program cost and the hats are estimates. This is not advertised as A^* with an admissible heuristic. Reserve a nonzero scheduling share for cost-layered grammar enumeration, and another for unresolved verification work. An initial engineering default can give one in eight quanta to the baseline lane, subject to measurement; no theoretical claim depends on that specific ratio.

Every primitive grammar expansion must have positive cost or belong to a bounded normalization phase. Increase provider windows, literal bounds, type-instantiation grammars, and recursion-plan bounds through explicit layers. An infinite sequence of zero-cost variants would defeat both minimal-cost enumeration and fairness.

13.2 A useful conditional completeness statement

Definition 13.1. *Fix a query and an explicit effective grammar of construction plans and certificates. A solution is replayable if it has a finite plan whose native reconstruction and final checks succeed using finite resources under the fixed environment and policy.*

Proposition 13.2 (Proof-relative eventual discovery). *Assume that each cost layer contains finitely many plans; every finite plan and retained alternative is eventually scheduled with sufficient finite reconstruction resources; pruning is certified or preserves a fair regeneration path; and verification fairly explores the grammar’s certificate plans. Then every replayable solution is eventually discoverable in an unbounded run.*

Proof. Take a finite successful program-and-certificate plan. It occurs in a finite cost layer and cannot be permanently lost through the permitted pruning or retention operations. By fairness, its finite prefixes and reconstruction attempts are eventually executed with the required resources. The certificate is similarly reached. The publication checks then accept the original-query judgments. The assumptions, rather than heuristic priority scores, provide the guarantee. $\square$

This is not a completeness theorem for a fixed timeout, for all Lean terms, or for the attached prototype. It is a design contract for the baseline lane. A grammar missing the needed recursor, literal, theorem, universe instantiation, or invariant cannot discover that solution, regardless of its scheduler. Report the grammar and bound with every coverage claim.

13.3 Cache identities must include the hidden context

A sound reuse key contains the environment generation; the closed target and specification; the ordered dependent telescope including local let-values and local instances; universe parameters and relevant constraints; inherited metavariable ownership and reachable assignments; transparency and elaboration options; grammar identity; and semantic-domain versions. Example-dependent caches also contain an observation-set generation.

Do not use pretty-printed types as cache keys. Do not globally union native unification successes as though they were state-independent equalities: a successful comparison may have changed the constraint store. Syntactic hash-consing of immutable expressions is useful, but semantic reuse requires the corresponding context and reconstruction checks.

A successful local solution can be abstracted over its actual dependency telescope and reused as a fresh closure. Instantiate its parameters and universes anew, and check its applicability. The abstraction

must preserve local let semantics and must not leave dangling free-variable identifiers. A negative cache entry has a narrower meaning: a particular bounded operation failed under a specific state. Reusing that entry as global non-inhabitation is prohibited.

13.4 Parallelism and memory

Parallelize alternative plans or independent closed verification tasks first. Do not let several workers mutate one native metavariable context. Sibling goals connected by assignments remain one correlated transaction unless a precise independence analysis establishes that their assignment sets and dependencies are disjoint.

A worker result is a closed proposal or a replay plan, not a raw foreign `MVarId`. The parent reconstructs or instantiates it under the current query and rechecks it before committing. If the parent context has changed, the old result is stale rather than an invitation to merge incompatible metavariable states.

Measure checkpoint retention, expression sharing, cache hit rate, proof-term size, and replay cost. Bound the hot native-state frontier and evict cold branches to exact recipes. These measurements determine whether a persistent-snapshot or checkpoint-plus-replay profile wins for a workload; neither language folklore nor the performance of the tiny prototype decides that question.

14 Negative answers and the publication boundary

14.1 Three negative claims that must not be conflated

A complete propositional search engine is valuable, but its result describes a particular logical fragment. Dyckhoff’s contraction-free calculus supplies a basis for a terminating propositional lane; the interpretation of its atoms and permitted premises must be explicit.[15, 4]

Failure of intuitionistic proof search for excluded middle is not a Lean proof of its negation. In a policy allowing Lean’s standard classical principles, the proposition has an inhabitant. Even in a constructive policy, underivability of a particular formula is different from deriving its negation. A logical-fragment certificate should therefore produce `FragmentUnderivable` unless an additional checked theorem transports it to the actual negative claim.

Similarly, exhausting a provider grammar that omits a constructor says nothing about inhabitation in the full environment. A finite-grammar exhaustion certificate needs a checked enumeration/coverage argument for that grammar and checked rejection of its possible candidates. An empty heuristic queue alone is reported as an exhausted frontier within `Unknown`, not upgraded to a certificate.

A genuine logical refutation is a separate Lean proof. The accompanying file checks:

Listing 7: Checked actual non-inhabitation proof.

```
theorem noUniversalChooser : ¬ Nonempty ∀( α : Type, α) := by
  intro existsChooser
  obtain () choose := existsChooser
  exact (choose Empty).elim
```

This proof specializes a hypothetical chooser at an actually empty type. It does not infer impossibility from a failed search. A negative lane can attempt such proofs directly, including structural contradiction and specification inconsistency, while ordinary synthesis continues.

14.2 Publication is a transaction against the original problem

The final checker should perform the following sequence as one controlled operation.

1. Load the frozen original query and the candidate’s complete auxiliary-declaration closure. Reject stale environment generations and undeclared dependencies.
2. Instantiate assigned metavariables and reject every remaining term or level metavariable, dangling free variable, or loose bound variable. Close the implementation and property over the original telescope in dependency order.
3. Construct declarations whose types are the original closed target and the original closed property applied to the exact proposed implementation. Do not substitute the candidate’s inferred statement for either one.
4. Submit the declarations through a kernel-checking path in a controlled environment, with checking bypasses disabled. Check auxiliaries in dependency order and forbid candidate-originated arbitrary axioms.
5. Audit transitive axiom dependencies and executable-policy requirements, then atomically publish the implementation, theorem, dependency report, and provenance.

Lean’s proof-validation guidance and axiom-reporting mechanisms provide the existing foundation for these checks. Merely constructing an `Expr` or successfully running an elaborator is not the whole validation contract.[13, 12]

Proposition 14.1 (Publication soundness relative to the environment). *Assuming correctness of the kernel and the permitted imported environment, a publication checker following the sequence above publishes only terms satisfying (1), under its reported assumptions and axiom policy.*

Proof. The checker fixes the declarations’ types from the original query, closes them over the intended telescope, and requires successful kernel checking. The implementation declaration therefore establishes the first judgment; the specification declaration establishes the second for that exact implementation. The dependency audit excludes unpermitted assumptions introduced by proposals. Search heuristics and external solver verdicts do not occur as additional premises. $\square$

The theorem does not say that imported axioms are true, that the native compiler is verified, or that the stated specification captures the user’s unstated intent. Those are distinct assumptions. This is a small and auditable trust claim, rather than a blanket assertion that anything written in Lean is correct.

14.3 Axiom policy and executability are separate axes

Provide a minimal-axiom profile with an explicit whitelist, a standard mathematical proof profile, and optional project-specific assumptions. Always reject `sorryAx` in finished certified artifacts. A native-evaluation proof mechanism can add computation-specific axioms and must not silently pass as a kernel-only reconstructed proof. Inspect actual dependencies rather than relying on a tactic’s familiar name.[12, 13]

Separately require that generated computational definitions are executable under the requested profile. Classical reasoning in erased proofs need not prevent a program from compiling, whereas classical choice used to produce runtime data can. An axiom whitelist alone is not an executability check. Likewise, an unsafe replacement implementation can behave differently from its logical model even when the

logical theorem is valid. A runtime-conformance profile must audit replacement mechanisms and state its compiler and foreign-code assumptions.[12, 10]

For normal code generation, the recommended default is total executable output, ordinary reconstructed proofs under an explicit standard axiom whitelist, and no unreviewed native-evaluation axioms. More restrictive or noncomputable modes should be explicit query options, not accidental consequences of which proof tactic ran first.

15 Implementation structure and user-facing integration

15.1 A concrete module decomposition

The following modules are proposed interfaces, not a list of implemented files. The narrow checked surfaces should hide their constructors and expose inspection views separately from validated values.

Module	Responsibility and invariant
<code>Leant2.Query</code>	Elaborated query, universe/telescope closure, inherited metavariable ownership, grammar and policy identities. Owns the immutable original problem.
<code>Leant2.Native</code>	Version-specific adapter for native goals, context entry, application, reduction, cases, recursors, instance resolution, and complete state rollback.
<code>Leant2.Plan</code>	Pure construction recipes and explicit continuations. Stores no competing Lean type calculus and no serializable raw local IDs.
<code>Leant2.Search</code>	Jobs, fair/heuristic queues, nonrefundable budgets, state materialization, checkpoints, and progress events.
<code>Leant2.Rules</code>	Introduction, head-spine construction, exact projections, dependent case plans, and controlled higher-order instantiations.
<code>Leant2.Recursion</code>	Carrier/generalization selection, structural method holes, invariant candidates, and well-founded call evidence.
<code>Leant2.Library</code>	Environment indexes, staged retrieval, provider roles, and exact-occurrence contract instantiation.
<code>Leant2.Behavior</code>	Observation banks, admissible inputs, verifier outcomes, CEGIS coordination, and reversible parking.
<code>Leant2.Domain</code>	Reified domain interfaces, checked interpretation theorems, summary applicability, and completion-wide rejection certificates.
<code>Leant2.Solver</code>	Optional solver process owner, finite sketch encodings, model decoding, proof reconstruction dispatch, and timeout/error separation.
<code>Leant2.Check</code>	Closure validation, original-statement kernel checking, auxiliary dependency audit, axiom/executable policy, and atomic publication.
<code>Leant2.Frontend</code>	Tactics, commands, editor actions, and standalone request presentation. Contains no independent synthesis semantics.
<code>Leant2.Worker</code>	Toolchain-matched worker protocol, environment generations, cancellation, restart, and durable replay.

The engine needs four especially important interface contracts. A proposal cursor produces resumable construction choices without claiming success. A native executor runs a choice transactionally and returns its complete assignment and obligation effects. A behavioral domain either returns a checked certificate with an applicability witness or an explicitly heuristic result. The publisher alone constructs a `CertifiedResult`. Making these contracts precise early is more important than assigning a large number of implementation modules.

15.2 Frontends share one request model

Proposed surface forms include `by leant2` for a goal, `leant2?` for a replayable suggestion, and a command such as:

Listing 8: Proposed command syntax, not implemented by the attached prototype.

```
#synth appendLike : List Nat → List Nat → List Nat
  where ∀ xs ys, appendLike xs ys = xs ++ ys
  using [List.nil, List.cons, List.rec]
```

The command elaborates the named function only as a specification binder until synthesis succeeds. It must not insert an axiom of the requested type as a provider. On success it emits a definition and a theorem; on failure it leaves neither a partial definition nor a trusted placeholder in the environment.

The exact spelling is secondary to the shared API: prepared query in, structured event stream out, checked publication or qualified termination result. An editor can display the best typed candidate, unresolved proof obligations, current recursion plan, and remaining search budget. A CLI can serialize the same information. Generated source suggestions should be re-elaborated against the same statement before being advertised as copy-paste-ready.

15.3 Lean-native does not require one process

Implement the semantic engine and frontends in Lean. A standalone Lean controller should launch a project-toolchain-matched worker under the project’s Lake environment. The worker owns native environments and checkpoints. A tactic can also run the library in-process, while using the same query preparation and publication code. Toolchain isolation is an architectural requirement independent of the implementation language.[6]

Protocol messages should contain a version, query ID, environment generation, cancellation token, operation, and bounded payload. A worker-local capability is valid only for its owning generation. Persistent state consists of replayable project/session commands and closed artifacts, not raw environment handles. A timeout kills or cancels a worker-owned operation without committing partial declarations.

External solvers are optional services, not a required second implementation of synthesis. Ordinary process launching can use Lean’s process APIs. If a deployment requires the descriptor-bound executable-identity guarantees of Djex’s existing Linux launcher, an optional narrow platform shim can preserve that policy; this does not justify retaining Haskell in the semantic architecture.[5]

Metaprogram plugins can execute host code even though their logical proposals are untrusted. Use process/resource isolation appropriate to the plugin trust level, and do not let an untrusted proposer mutate the publisher’s baseline environment. The logical trust boundary and the operating-system execution boundary solve different problems.

15.4 Diagnostics are part of the design

A useful failure report identifies the blocked obligation and attempted strategies: unresolved family instantiation, insufficient recursion generalization, failed decrease proof, unavailable contract lemma, rejected axiom, stale cache, or exhausted grammar layer. It should include the relevant native type and the chosen argument/universe/dictionary assignments.

Record deterministic replay identifiers, proposal counts, branch restorations, proof attempts, solver outcomes, summary rejections, and parked-member reactivations. A large trace should be demand-driven; the default report should show the shortest causal explanation and one or two promising residual plans. This information makes the synthesizer debuggable as a semantic engine rather than a black box that occasionally prints a term.

16 Coverage targets and an implementation sequence

16.1 Operationalize “a large fraction of practical Lean types”

The target is broad practical usefulness, not an unsupported percentage. Measure it on a pinned, held-out corpus with explicit grammars and specifications. Separate ordinary inhabitation tasks from behavioral tasks, library reuse from implementation discovery, and executable code from noncomputable mathematical witnesses.

Family	Principal route	Important remaining difficulty
Function and record plumbing	Focused Pi introduction, locals, spines, projections	Search ranking and dictionary identity
Higher-rank callbacks	Native dependent Pi structure and demanded instantiation	Higher-order/universe search remains bounded
Subtypes and dependent pairs	Shared witness/proof constraints and continuation search	Invariant or arithmetic discovery
Indexed datatypes and finite indices	Native motives, cases, index equations	Generalization and nondefinitional index equalities
List/tree transformations	Library combinators and structural certificate algebras	Carrier and invariant selection
Arithmetic and bit-level functions	Typed sketches plus checked domain procedures	Exact casts, overflow, and proof reconstruction
Well-founded algorithms	Call skeleton plus decrease and behavior obligations	Measure selection and helper invariants
Quotient-based mathematics	Existing lifts, eliminators, and respectfulness lemmas	Computational representatives cannot be invented freely
Effectful, partial, and coinductive code	Registered semantic profiles and specialized plans	Requires an explicit effect or fixpoint specification

Build the corpus from three sources: a snapshot of existing Leant/Djex regression signatures; new dependency-sensitive microcases that the old fragment cannot exploit; and real project tasks with

behavior stated independently of a hidden implementation. Keep task families together when splitting development and held-out sets, so syntactic variants do not masquerade as generalization.

For a from-scratch task, inspect the allowed inventory for answer leakage, including aliases, wrapper functions, and imported theorems that simply expose a ready-made implementation. For a reuse task, such providers are a feature, not cheating. Report those two regimes separately. Neither historical Church-signature counts nor this article’s tiny examples support a claim about a production success rate.

16.2 Metrics and adversarial controls

Report time to first certified implementation, fraction certified under fixed budgets, typed-only and tested-only outcomes, proof-search unknowns, memory high-water mark, native reconstruction failures, proof size, and declared dependencies. Count solver calls, replayed counterexamples, certified partial-program rejections, and reactivated parked candidates. Keep wall-clock latency separate from deterministic work counters.

Ablate native dependency information, whole-continuation backtracking, projection ordering, recursion-carrier discovery, proof-directed contracts, summary pruning, reversible parking, and retrieval widening. An apparent performance gain is not acceptable if it silently weakens the specification or changes a certified result into a tested-only result.

The acceptance suite must contain deliberate failures: an unsatisfiable target, a satisfiable target outside a restricted grammar, conflicting local dictionaries, independent universes, dependent sibling holes, a stale cache after a context change, forbidden axioms hidden in helpers, solver `unknown`, a malformed model, a worker timeout, and a source renderer that changes an implicit argument. These tests protect the claims made by the interface, not just the number of successful examples.

16.3 Milestones with semantic gates

M0: query and publication foundation. Implement strict preparation, environment generations, original-statement checking, closure, transitive dependency audits, and structured outcomes. Gate: fabricated candidates, unresolved holes, stale contexts, and forbidden axioms cannot be published.

M1: native nonrecursive construction. Implement Pi introduction, spines, projections, constructors, continuation rollback, and a bounded fair scheduler. Gate: type-only baseline tasks plus the dependent-witness and dictionary controls pass with stable diagnostics.

M2: native dependency and library scale. Add general dependent cases, universe-aware retrieval, type-class integration, and context-correct caching. Gate: indexed and multi-universe tasks, local-instance shadowing, and replay across workers preserve the same statements and selected values.

M3: recursive certificate construction. Add carrier generalization, structural algebras, and then well-founded call plans. Gate: nontrivial held-out recursive functions receive universal certificates; bounded unrolling is never mislabeled as a general implementation.

M4: behavioral refinement. Add verified domain summaries, CEGIS, finite typed sketches, and reversible parking. Gate: each permanent prune has an applicable certificate; unknown solver/prover outcomes cannot reject valid candidates or block fair progress.

M5: production integration and performance. Add editor/CLI polish, memory-tiered checkpoints, controlled parallelism, and toolchain adapters. Gate: measured gains preserve the M0–M4 semantic controls and the same pinned-query results.

These stages are an implementation order, not a recommendation to retain a Haskell engine permanently. The successor’s architecture is Lean-native from M0; existing systems serve as regression references and sources of test problems.

17 Experiments actually performed

17.1 Environment and verification method

The local working container had no Lean toolchain and no usable network path. The supplied tools described AXLE’s remote check API. The checks were made synchronously through the Wolfram connector’s HTTP facilities, preserving `import Lean`, with `ignore_imports = false` and `theorems_only = false`. The service reported Lean 4.34.0. No claim is made that this was the latest Lean release.[20]

Both accepted source files were checked in full, including definitions, anonymous expected-failure controls, and `##print axioms` commands. The processed-source echo matched each submitted source with exactly one additional terminal line feed. It was not byte-for-byte identical, and the imports were not replaced. The accompanying JSON files are structured transcriptions of the actual returned evidence, not raw HTTP captures. The replay client saves raw responses on new runs.

The service returned generic advisories recommending its cached Mathlib import header. Those were not compiler errors and were not used to change the experiment. It did not report a Mathlib commit; the experiment’s source imports only Lean. The reported executor identifiers and request IDs are retained for provenance, but they are not a substitute for rebuilding or independently checking the files.

17.2 The native search experiment

`prototype/NativeCore.lean` contains a 178-line self-contained experiment, including its tests. The search implementation operates directly on Lean goals, saves branch state, invokes native application and case analysis, discovers constructor names from the environment, and generates exact structure projections. It uses a depth-first continuation strategy with path fuel 64 and a split allowance of 3.

Checked definition	Purpose	Axioms
<code>identitySort</code>	Sort-polymorphic identity	None
<code>composeDependent</code>	Composition between indexed fibers	None
<code>applicationDependent</code>	Apply a dependent function at its argument	None
<code>usePolymorphic</code>	Consumer with a polymorphic callback signature	None
<code>useTwoUniverses</code>	Two callback domains with independent universes	None
<code>swapSum</code>	Sum case analysis and reconstruction	None
<code>swapProd</code>	Product reconstruction from actual fields	None
<code>dependentWitness</code>	Witness constrained by a later equality	None
<code>transport</code>	Equality transport between dependent fibers	None
<code>vectorHead</code>	Elimination of an indexed nonempty vector	<code>propext</code>
<code>vectorReplicate</code>	Hand-selected induction; synthesized branch bodies	None
<code>chooseDictionary</code>	Preserve the selected dictionary payload	None

There are eleven whole-goal synthesized definitions and one manually chosen recursion template. Two

additional `rfl` theorems check the actual witness and dictionary result, not merely their types. Two `fail_if_success` controls require the bounded tactic to fail on `False` and on a universal chooser type. These controls check search behavior; they are not the prototype proving non-inhabitation.

The final service response had `okay = true`, no Lean errors, no tool errors, and no failed declarations. Fourteen named axiom inventories were requested: thirteen were empty, and `vectorHead` used `propext`. None contained `sorryAx`. Compiler warnings concerned two intentionally unused binders and the two expected-failure controls.

The accepted request identifier is

217a1c39-4a72-4447-adf2-69426f4439b8.

The service reported 1,408 ms for its file-processing total and 1,527 ms for the whole request. These are not isolated synthesis timings, not a repeatability study, and not a comparison against Leant or Djex.

17.3 The initial failure and what changed

The first native run exhausted the default 200,000-heartbeat limit while trying the dictionary witness. Constructor growth was ahead of structure-field discovery. The revised algorithm added projection proposals before constructors and propagated heartbeat/interrupt exceptions instead of treating them as ordinary backtrackable rule failures.

That earlier file did not pass. Its missing dictionary definition also caused a downstream failed theorem with a `sorryAx` dependency. Those diagnostics are not evidence against the corrected final file, but they are useful evidence for the design: branch ordering and failure classification must be tested rather than inferred from a high-level architecture diagram. The final receipt supersedes that failed run; the experiment history records both.

17.4 Checked reference certificates

`prototype/Certificates.lean` is a separate, manually authored file. Its recursive append construction and accompanying correctness theorem were accepted without axioms. The actual universal-chooser refutation was also accepted without axioms. The completion-wide prefix rejection theorem was accepted with `propext` and `Quot.sound`.

The accepted request identifier is

d599bef4-fec8-4a64-b2b1-e09b3fa622b0.

The file had no Lean errors or warnings and no failed declarations. The two generic service-header advisories remained. Two earlier drafts of this reference file had ordinary syntax/API-use errors, corrected before the accepted run; they are documented in the experiment history. These certificates validate the stated constructions, not an implemented automatic recursion or pruning engine.

17.5 Finite behavioral laboratory

`experiments/behavior_lab.py` uses only Python's standard library. It enumerates the grammar

$$t ::= [] \mid x \mid y \mid \text{reverse}(t) \mid \text{append}(t, t)$$

through seven AST nodes. Inputs are pairs of binary-valued lists of length at most two. Each term has an exact affine length interpretation

$$|\llbracket t \rrbracket(x, y)| = a_t|x| + b_t|y|.$$

The interpretation follows by structural induction over this grammar: leaves supply $(0, 0)$, $(1, 0)$, or $(0, 1)$; reversal preserves the coefficients; append adds them. The Python implementation was tested on all generated terms and all concrete input pairs. It was not formalized in Lean.

Measurement	Result
Terms enumerated	1,875
Concrete input pairs	49
Checks of the affine-length interpretation	91,875
Terms with append-compatible coefficients $(1, 1)$	382
Terms excluded by the necessary length condition	1,493
Terms passing every finite append observation	89
Observation buckets as three examples accumulate	1, 2, 8
Retained members after parking/refinement	382
Mechanism unit tests passed	5 of 5

Table 6: Deterministic finite-laboratory results, not production coverage or speed measurements.

The necessity of $(1, 1)$ here relies on the laboratory’s actual inhabited element domain, where lists of lengths zero and one exist. A generic abstract domain for arbitrary Lean types must preserve such realizability assumptions. This small example is not permission to invent nonempty inputs at an arbitrary fixed type.

The five unit tests cover affine-length evaluation, recovery of a parked candidate after a distinguishing example, the difference between refuting one completion and a whole sketch, nonblocking handling of verification-unknown, and a toy correlated-witness choice. The first append candidate already passes this laboratory’s finite tests, so its recorded CEGIS trace contains no nontrivial counterexample iteration. The experiment does not claim a measured CEGIS speedup or a universal append proof from testing. Append is an explicit primitive in this grammar.

17.6 What remains unimplemented

The prototype has no global provider index, fair portfolio scheduler, automatic recursion-carrier discovery, SMT integration, certified abstract-domain plugin, persistent cache, or standalone publication service. Its fuel bounds a continuation path, not total work. It deliberately skips arbitrary natural-number case splitting and its case-proposal guard also omits empty inductives; it is not a general dependent elimination tactic. It relies on the surrounding Lean declaration checker for the tested files, rather than implementing the proposed independent publication transaction.

These boundaries are why the archive calls it a native-core experiment. Its value is that it exercises real Lean expressions, universes, dependent contexts, projections, rollback, and kernel-checked output instead of simulating them in another language. The architecture specifies how to grow from those mechanisms without inheriting the existing cross-language type approximation.

18 Conclusion

Leant2’s principal opportunity is not to make Haskell-style inhabitation search more elaborate. It is to let the search use the dependency information that Lean already possesses, and to make behavioral proof obligations participate in construction from the beginning.

The resulting design has a native semantic core, explicit correlated continuations, typed recursion plans, graded behavioral evidence, and one final check against the original query. It retains bounded and specialized search where that is useful, while exposing the bounds honestly. The most important performance mechanisms—library retrieval, carrier discovery, abstract pruning, and observational parking—are designed so that heuristic convenience does not silently become a claim of equivalence, completeness, or impossibility.

The checked experiments support a concrete starting point. The next engineering objective is to turn the small native engine into a replayable obligation scheduler and publication API, then add recursive certificate algebras and certified behavioral domains. That path directly serves Lean programs and proofs; it needs no Haskell frontend, Haskell type approximation, or compatibility obligation.

A Complete checked native-core experiment

The following is the complete `NativeCore.lean` submitted in the accepted run. It is small enough to audit, but it is not the production architecture. The source in the archive preserves Unicode and can be checked with the pinned toolchain. Line wrapping below is only for presentation.

```

1 import Lean
2
3 open Lean Meta Elab Tactic
4
5 /-!
6 A deliberately small, bounded Leant2 experiment, not the proposed full engine.
7 It searches native Lean goals, uses transactional continuation backtracking,
8 and discovers constructors and case splits from the actual environment.
9 It has no external solver, library index, recursion discovery, or negative oracle.
10 -/
11 namespace Leant2Mini
12
13 private def attempt (action : MetaM Bool) : MetaM Bool := do
14   let saved ← saveState
15   try
16     if ← action then return true
17   catch error =>
18     saved.restore
19     if error.isInterrupt || error.isMaxHeartbeat then throw error
20   saved.restore
21   return false
22
23 private def constructors (type : Expr) : MetaM (List Name) := do
24   let type ← whnf type
25   match type.getAppFn with
26   | .const name _ =>
27     match ←( getEnv).find? name with
28     | some (.inductInfo info) => return info.ctors
29     | _ => return []
30   | _ => return []
31
32 /-- Fuel bounds a complete continuation path, not total work or elapsed time.
33 A successful choice must solve every remaining goal before it commits. -/
34 private partial def solve (fuel splits : Nat) (goals : List MVarId) : MetaM Bool := do
35   match goals with
36   | [] => return true
37   | goal :: rest =>
38     if ← goal.isAssigned then return ← solve fuel splits rest
39     match fuel with
40     | 0 => return false
41     | fuel + 1 => goal.withContext do
42       let target ← whnf ←( goal.getType)
43       if target.isForall then
44         return ← attempt do
45           let (_, body) ← goal.intro1
46           solve fuel splits (body :: rest)
47       if ← attempt do
48         goal.refl
49         solve fuel splits rest

```

```

50   then return true
51   let mut locals : Array FVarId := #[]
52   for decl in ←( getLCtx) do
53     if !decl.isImplementationDetail then
54       locals := locals.push decl.fvarId
55   for localId in locals do
56     if ← attempt do
57       let children ← goal.apply (mkFVar localId) { newGoals := .all }
58       solve fuel splits (children ++ rest)
59     then return true
60   -- Projection proposals retain the exact dictionary/structure value.
61   -- They must precede recursive constructor growth (e.g. Nat.succ).
62   for localId in locals do
63     let localType ← whnf ←( inferType (mkFVar localId))
64     if let .const name _ := localType.getAppFn then
65       if let some info := getStructureInfo? ←( getEnv) name then
66         for index in [:info.fieldNames.size] do
67           if ← attempt do
68             let projection := Expr.proj name index (mkFVar localId)
69             let children ← goal.apply projection { newGoals := .all }
70             solve fuel splits (children ++ rest)
71           then return true
72   for name in ←( constructors target) do
73     if ← attempt do
74       let constructor ← mkConstWithFreshMVarLevels name
75       let children ← goal.apply constructor { newGoals := .all }
76       solve fuel splits (children ++ rest)
77     then return true
78   if splits > 0 then
79     for localId in locals do
80       let localType ← whnf ←( inferType (mkFVar localId))
81       -- Arithmetic induction is outside this experiment. Avoid unhelpful
82       -- repeated splitting of arbitrary natural-number parameters.
83       if !localType.isAppOf ``Nat && ←( constructors localType).isEmpty then
84         if ← attempt do
85           let branches ← goal.cases localId
86           let children := branches.toList.map (fun branch => branch.mvarId)
87           solve fuel (splits - 1) (children ++ rest)
88         then return true
89   return false
90
91 elab "leant2_mini" : tactic => do
92   let saved ← saveState
93   let goals ← getGoals
94   if ← solve 64 3 goals then
95     setGoals []
96   else
97     saved.restore
98     throwError "Leant2Mini: bounded search found no term (not a non-inhabitation result)"
99
100 end Leant2Mini
101
102 universe u v w
103 namespace Leant2Experiments
104

```

```

105 def identitySort α( : Sort u) : α → α := by leant2_mini
106
107 def composeDependent α{ : Sort u} {P Q R : α → Sort v}
108   (f : ∀ x, Q x → R x) (g : ∀ x, P x → Q x) :
109   ∀ x, P x → R x := by leant2_mini
110
111 def applicationDependent α{ : Sort u} β{ : α → Sort v}
112   (f : ∀ x, β x) (x : α) : β x := by leant2_mini
113
114 def usePolymorphic (consume : ∀( α : Type u, α → α) → Nat) : Nat := by
115   leant2_mini
116
117 def useTwoUniverses {R : Type w}
118   (consume : ∀( α : Type u, α → α) → ∀( β : Type v, β → β) → R) : R := by
119   leant2_mini
120
121 def swapSum α{ : Type u} β{ : Type v} : Sum α β → Sum β α := by leant2_mini
122
123 def swapProd α{ : Type u} β{ : Type v} : α × β → β × α := by leant2_mini
124
125 def dependentWitness (a b : Nat) : {n : Nat // n = b} := by leant2_mini
126
127 -- This checks the chosen computational witness, not merely its type.
128 theorem dependentWitness_value (a b : Nat) : (dependentWitness a b).val = b := by rfl
129
130 def transport α{ : Type u} {P : α → Type v} (a b : α)
131   (equality : a = b) (value : P a) : P b := by leant2_mini
132
133 inductive Vec α( : Type u) : Nat → Type u where
134   | nil : Vec α 0
135   | cons {n : Nat} : α → Vec α n → Vec α (n + 1)
136
137 def vectorHead α{ : Type u} {n : Nat} : Vec α (n + 1) → α := by leant2_mini
138
139 -- Recursion skeleton selected by hand; only its branch bodies are synthesized.
140 def vectorReplicate α{ : Type u} (n : Nat) (a : α) : Vec α n := by
141   induction n with
142   | zero => leant2_mini
143   | succ n smaller => leant2_mini
144
145 class Token α( : Type u) where
146   value : α
147
148 def chooseDictionary (first second : Token Nat) : {n : Nat // n = second.value} := by
149   leant2_mini
150
151 theorem chooseDictionary_value (first second : Token Nat) :
152   (chooseDictionary first second).val = second.value := by rfl
153
154 -- Expected failures are asserted, not left as failing declarations or sorries.
155 example : True := by
156   fail_if_success have impossible : False := by leant2_mini
157   trivial
158
159 example : True := by

```

```
160   fail_if_success have impossible :  $\forall(\alpha : \text{Type}, \alpha) := \text{by leant2\_mini}$ 
161   trivial
162
163 #print axioms identitySort
164 #print axioms composeDependent
165 #print axioms applicationDependent
166 #print axioms usePolymorphic
167 #print axioms useTwoUniverses
168 #print axioms swapSum
169 #print axioms swapProd
170 #print axioms dependentWitness
171 #print axioms dependentWitness_value
172 #print axioms transport
173 #print axioms vectorHead
174 #print axioms vectorReplicate
175 #print axioms chooseDictionary
176 #print axioms chooseDictionary_value
177 #eval Lean.versionString
178 end Leant2Experiments
```

B Representative production invariants and regression tests

Invariant	Minimum regression test
Original statement is immutable	A candidate specialized to equal universes must not satisfy a query with two independent universe parameters.
Continuation choices are reversible	The first well-typed witness violates a later dependent predicate; a second witness succeeds.
Local dictionaries are values	Two same-typed dictionaries with different payloads remain distinguishable in both output and certificates.
Case motives preserve dependency	An indexed constructor changes the type of a later local declaration; branch reconstruction must use the specialized declaration.
Recursion hypotheses are restricted	A proposed self-call at the same major argument cannot acquire an unrestricted recursive provider.
Generalization is behavioral as well as type-directed	Lookup in a list needs the tail hypothesis at a changed index.
Tests do not prove universals	A candidate matching all initial inputs but failing a later admissible input never receives a universal status.
Pruning covers all completions	A failed empty-tail completion cannot reject a sketch whose nonempty-tail completion is correct.
Parking is reversible and live	Two candidates agree initially; after refinement or fair member rotation, the previously parked candidate is explored.
Negative outcomes are qualified	Missing all providers, proof timeout, logical-fragment underivability, and checked non-inhabitation have different result constructors.
Caches include semantic context	Reusing a result after changing a local let-value or instance selection forces applicability checking or invalidation.
Publication audits auxiliaries	A helper depending on <code>sorryAx</code> cannot hide behind a short final theorem.
Cancellation does not refund work	Repeated restore/retry operations cannot evade a global rule or solver budget.
Worker capabilities have generations	A completed task from a replaced environment cannot commit into the new environment.

C Artifact map and reproducibility

`article/Leant2.tex` is the master source. Its neighboring numbered `.tex` files contain the article and references. Build with three XeLaTeX passes from a clean source directory to stabilize the contents and cross-references. The archive's Makefile provides the same operation. Latin Modern fonts are standard TeX dependencies; the monospace configuration prefers DejaVu Sans Mono when available and otherwise uses Latin Modern Mono. No font binaries are included.

`prototype/NativeCore.lean` and `prototype/Certificates.lean` are the exact accepted sources. `prototype/lean-toolchain` selects Lean 4.34.0. With that toolchain installed, run `lean NativeCore.lean` and `lean Certificates.lean` from the prototype directory. The optional Python remote-check client requires network access and records a fresh raw response; the actual checks for this article used Wolfram's HTTP transport instead.

`experiments/native_core_receipt.json` and `experiments/certificates_receipt.json` preserve structured summaries of the successful remote responses, including source-echo qualifications, axioms, timings, and executor identity. `experiments/HISTORY.md` records failed preliminary runs and their corrections. These files are evidence reports, not independently signed attestations.

`experiments/behavior_lab.py` regenerates `behavior_results.json` and runs the five finite mechanism tests. The script has no third-party Python dependency. Its results are intentionally classified as finite executable checks rather than Lean proofs. The architecture, solver integration, scheduling theorem assumptions, and production module interfaces remain design artifacts.

References

- [1] Vladimir Reshetnikov and contributors. *Leant*. Source snapshot 6bf05ad78c467989e68290f2d08bbbed40802d485, inspected September 17, 2026. <https://github.com/VladimirReshetnikov/Leant/tree/6bf05ad78c467989e68290f2d08bbbed40802d485>.
- [2] Leant contributors. *Native-to-fragment translation*, `src/Leant/Synth/Fragment.hs`, especially the module documentation and fragment constructors, at the Leant snapshot above. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/src/Leant/Synth/Fragment.hs>.
- [3] Leant contributors. *Synthesis internals*, `docs/synth-internals.md`, at the same snapshot. Exact-origin, candidate-ownership, and behavioral evidence boundaries. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/docs/synth-internals.md>.
- [4] Vladimir Reshetnikov and contributors. *Djex README*. Default-branch documentation inspected September 17, 2026. <https://github.com/VladimirReshetnikov/Djex/blob/main/README.md>. Leant’s inspected submodule separately pins Djex commit e237e8667190aff38faaa590ce5b12afbfffac452; the documentation inspection is not a rerun of the historical tests it reports.
- [5] Djex contributors. *Djex architecture*, `docs/architecture.md`. Default-branch documentation inspected September 17, 2026. <https://github.com/VladimirReshetnikov/Djex/blob/main/docs/architecture.md>.
- [6] *Rewriting Leant and Djex in Lean 4*. Prior analysis retained in Leant, `docs/Leant_Djex_Lean4_Rewrite_Analysis/`, source inspected at the Leant snapshot above. Its proposed migration architecture is distinguished from the successor design in this article. https://github.com/VladimirReshetnikov/Leant/tree/6bf05ad78c467989e68290f2d08bbbed40802d485/docs/Leant_Djex_Lean4_Rewrite_Analysis.
- [7] Lean community. *The Lean 4 Metaprogramming Book*, chapter on **MetaM**. Documentation accessed September 17, 2026; individual APIs used by the prototype were also checked against the actual Lean 4.34.0 environment. https://leanprover-community.github.io/lean4-metaprogramming-book/main/04_metam.html.
- [8] Lean language reference. *The Type System*, including functions, propositions, and universes. Accessed September 17, 2026. <https://lean-lang.org/doc/reference/latest/The-Type-System/>.
- [9] Lean language reference. *Inductive Types*. Accessed September 17, 2026. <https://lean-lang.org/doc/reference/latest/The-Type-System/Inductive-Types/>.
- [10] Lean language reference. *Recursive Definitions*. Accessed September 17, 2026. The rolling documentation may describe facilities newer than the experiment’s pinned toolchain; the experiment makes no claim to test all such facilities. <https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/>.
- [11] Lean language reference. *The grind tactic*. Accessed September 17, 2026. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [12] Lean language reference. *Axioms*. Accessed September 17, 2026. Includes transitive dependency reports, native-evaluation assumptions, and the limits of unrestricted parametricity. <https://lean-lang.org/doc/reference/latest/Axioms/>.

- [13] Lean language reference. *Validating a Lean Proof*. Accessed September 17, 2026. <https://lean-lang.org/doc/reference/latest/ValidatingProofs/>.
- [14] Jannis Limperg and contributors. *Aesop*, project README, inspected September 17, 2026. Best-first rule-driven proof search for Lean. <https://github.com/leanprover-community/aesop>.
- [15] Roy Dyckhoff. Contraction-free sequent calculi for intuitionistic logic. *Journal of Symbolic Logic*, 57(3):795–807, 1992.
- [16] Nadia Polikarpova, Ivan Kuraj, and Armando Solar-Lezama. *Program Synthesis from Polymorphic Refinement Types*. arXiv:1510.08419, 2015. <https://arxiv.org/abs/1510.08419>.
- [17] Zheng Guo, Michael James, David Justo, Jiaxiao Zhou, Ziteng Wang, Ranjit Jhala, and Nadia Polikarpova. *Program Synthesis by Type-Guided Abstraction Refinement*. arXiv:1911.04091, 2019. <https://arxiv.org/abs/1911.04091>.
- [18] Xinyu Wang, Isil Dillig, and Rishabh Singh. *Program Synthesis using Abstraction Refinement*. arXiv:1710.07740, 2017. <https://arxiv.org/abs/1710.07740>.
- [19] Tristan Knoth, Di Wang, Nadia Polikarpova, and Jan Hoffmann. *Resource-Guided Program Synthesis*. arXiv:1904.07415, 2019. <https://arxiv.org/abs/1904.07415>.
- [20] Axiom Math. *AXLE: check tool documentation*. API documentation accessed September 17, 2026. Actual experiment requests and returned environment identities are recorded in the accompanying receipts. <https://axle.axiommath.ai/v1/docs/tools/check/>.